

Robotics
Course Project
Robotic Assembly of Trophy

Aanand Nataraj Satheesh Manimala - 202403949
Malcolm Ian D Souza - 202403948
Alessandro Gorla - 202400715
Vilma Delden - 202400698



September 5, 2026

TABLE OF CONTENTS

	Page
CHAPTER	
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Definition	1
1.3 Problem Definition	1
2 Method	2
2.1 Robotics concept	2
2.1.1 Gripper contact forces	2
2.1.2 Gripper contact pressures	2
2.1.3 Gripper Statics	3
2.1.4 Optimal design of the gripper	4
2.1.5 Robot arm	6
2.1.6 Manipulation and motion planning	7
2.1.7 Statics of the robot arm	7
2.2 Sensor concept	8
2.2.1 Strain gauge	8
2.2.2 Torque sensors of the robot arm	10
2.2.3 Ultrasonic sensor	11
3 Results	12
3.1 Robotics concept	12
3.1.1 Gripper fabrication	12
3.1.2 Gripper manipulation	13
3.1.3 Assembly operation	14
3.2 Sensor concept	14
3.2.1 Strain gauge	14
3.2.2 Torque sensors of the robot arm	15
3.2.3 Ultrasonic sensor	16
4 Conclusion	17

5	Discussion	18
6	Contributions	19

BIBLIOGRAPHY

Bibliography	20
------------------------	----

APPENDIX

A	Appendix	21
A.1	Appendix 1	21
A.2	Appendix 2	22
A.2.1	Program to control gripper	22
A.2.2	Program for assembly operation	24
A.2.3	Program for retrieving the torques from the robot	27

1 | Introduction

1.1 | Background and Motivation

Automation and robotics plays a vital role in modern society and is used in many sectors, primarily for production required to assemble parts. This project is focused on building a gripper, programming the robot arm and incorporating sensors to perform a specific assembly task that involves a trophy. Sensory feedback is given in order for the robot arm to respond dynamically in its precise motions.

The sensors incorporated in the project include: strain gauges, torque sensors and an ultrasonic sensor. These measure and respond to force applied by the gripper, required torque of the motors of the arm and distance between the fingers of the gripper respectively. By using these sensors the robot arm can move in a more precise motion and make sure the parts are moved in a safe way without damage or collisions.

The paper outlines the development of the robot arm from designing the gripper, to calculating the kinematics, motion planning to the results of the final assembly process.

1.2 | Problem Definition

The defined problem is to assemble a trophy, part by part, using a UR3e robot arm together with a modeled and 3D-printed gripper. The trophy is composed by parts with different dimensions and shape, the detailed list of these components is provided in Appendix 1.

1.3 | Problem Definition

2 | Method

2.1 | Robotics concept

2.1.1 | Gripper contact forces

The interaction between the parts of the trophy and the gripper's fingers have been analyzed considering both the contact force and the maximum contact pressure. The contact force is computed according to the vertical equilibrium of forces shown in Figure 2.1.

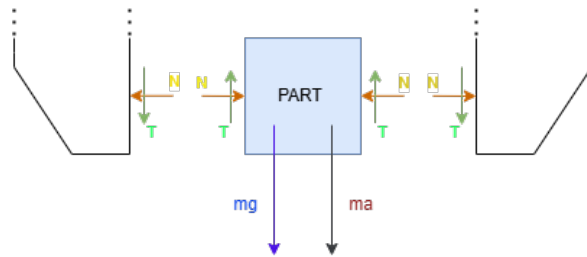


Figure 2.1: Contact force equilibrium

$$N = \frac{mg + ma}{\mu n} \quad (2.1)$$

m is the mass of the parts, g the gravity, a the maximum vertical acceleration of the robot arm that has been chosen as $a = 0.1m/s^2$, μ is the friction coefficient between the parts and the the finger and n is the number of fingers. Computing for each part the contact force, it has been found that the marble part requires the highest force in order to be lifted. This force is

$$F = 4.6082N$$

2.1.2 | Gripper contact pressures

The contact pressure has been computed for each part of the trophy, considering the different geometric contacts (surface contact for the base and nut, line contact for the rest of the parts) and the different materials. The surface contact and the line contact pressures have been computed respectively:

$$p = \frac{F}{a \cdot b} \quad (2.2)$$

$$p = 0.418 \cdot \sqrt{\frac{2 \cdot F \cdot E_r}{L \cdot d}} \quad (2.3)$$

a, b, L, d are the geometric dimension of the contacts.

F is the contact force previously computed.

E_r is the average coefficient of elasticity between the finger and the different parts of the trophy.

$$E_r = \frac{2E_{finger}E_{part}}{E_{finger} + E_{part}} \quad (2.4)$$

The maximum contact pressure has been found for each of the parts:

$$\begin{array}{llll} p_1 = 1.4591MPa & p_2 = 1.8kPa & p_3 = 0.4879MPa & p_4 = 0.2686MPa \\ p_5 = 1.2421MPa & p_6 = 0.1331MPa & p_7 = 1.2247MPa & p_8 = 0.5428MPa \\ p_9 = 0.7564MPa & p_{10} = 0.3kPa & & \end{array}$$

2.1.3 | Gripper Statics

The robot gripper will be 3D printed with a PLA filament, this makes the whole gripper very light. For this reason we decided to simplify the analysis considering only the static forces and not the dynamics of the gripper. The static relation between the contact force F_K and the torque applied by the motor on the link gear C as a function of the opening angle θ has been found. The gripper torque is found by drawing a free body diagram, as shown in figure 2.2.

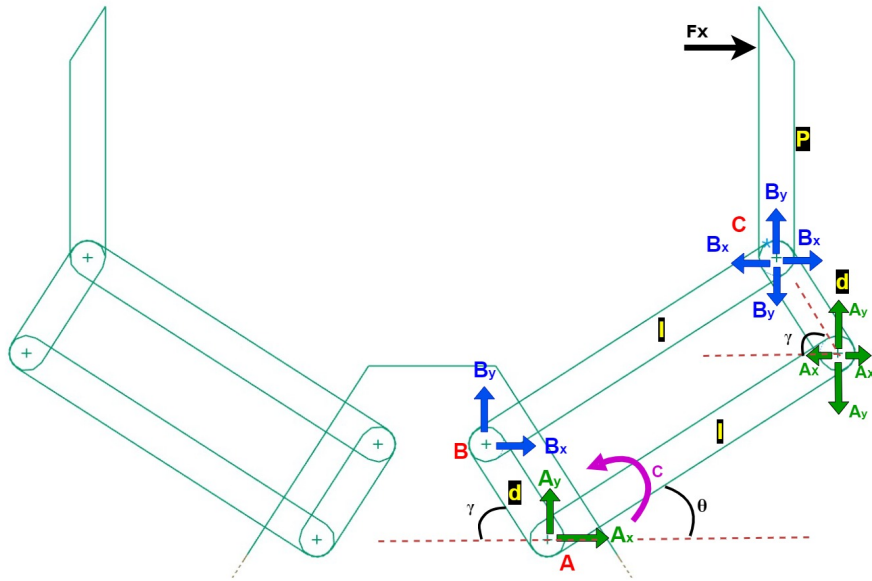


Figure 2.2: Free body diagram of the robot gripper

Link 1:

$$\sum M_A = C + A_x l \sin \theta - A_y l \cos \theta = 0 \quad (2.5)$$

Link 2:

$$\sum M_B = B_x l \sin \theta - B_y l \cos \theta = 0 \quad (2.6)$$

Link 3:

$$\sum F_x = A_x + B_x + F_x = 0 \quad (2.7)$$

$$\sum F_y = A_y + B_y = 0 \quad (2.8)$$

$$\sum M_C = F_K p + A_x d \sin \gamma + A_y d \cos \gamma = 0 \quad (2.9)$$

Combining momentum in link 1 (equation 2.5) and the sum of y-forces in link 3 (equation 2.8) forms equation 2.10. Moreover combining equation 2.10 and the sum of the momentum in link 2 (equation 2.6) forms equation 2.11. Furthermore, combining equation 2.11 and the sum of x-forces in link 3 (equation 2.7) finally forms equation 2.12.

$$C + A_x l \sin \theta + B_y l \cos \theta = 0 \quad (2.10)$$

$$C + A_x l \sin \theta + \frac{B_x \sin \theta}{\cos \theta} l \cos \theta = 0 \quad (2.11)$$

$$C + A_x l \sin \theta - (A_x + F_K) l \sin \theta = 0 \quad (2.12)$$

Some forces are canceled and the final equation for torque is given by equation 2.13.

$$C = F_K l \sin \theta \quad (2.13)$$

In order to avoid a zero-torque, the angle is limited between 10° and 90° .

2.1.4 | Optimal design of the gripper

The relation $C = F_K l \sin \theta$ is crucial for the design for the gripper. The parameter l is significant as minimizing its dimension reduces the torque. However, geometric constraints imposed by the parts must also be considered. The maximum diameter of the parts of the trophy with some extra tolerance is $d = 140mm$. At the open position of the gripper $\theta_{open} = 10^\circ$, a distance d is required between the fingers allowing the optimal value of l to be found.

$$l_{opt} = \frac{d}{2 \cos(\theta_{open})} = 72mm \quad (2.14)$$

Figure 2.3 shows the trend of the required torque to have the maximum contact force, depending on the opening of the gripper. The extreme values are $C_{max} = 331.8Nmm$ and $C_{min} = 57.6Nmm$

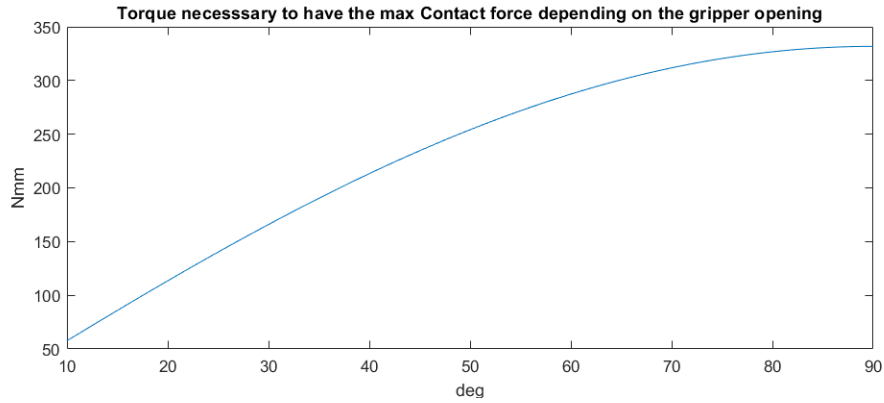


Figure 2.3: Torque on the first link to reach the maximum contact force depending on the opening of the gripper

Moreover the time required to complete all the movement passing from $\theta = 10^\circ$ to $\theta = 90^\circ$ has been set to $t = 3s$. The angular velocity of link 1 can be computed as

$$\dot{\theta} = \frac{\pi/2 - \pi/18}{t} = 0.4654 \text{ rad/s} \quad (2.15)$$

The horizontal velocity of the finger can be computed instead as:

$$v = l \sin(\theta) \dot{\theta} \quad (2.16)$$

Figure 2.4 shows the horizontal velocity of the finger depending on the opening position of the gripper, in a preliminary design estimation (considering $\dot{\theta}$ a constant).

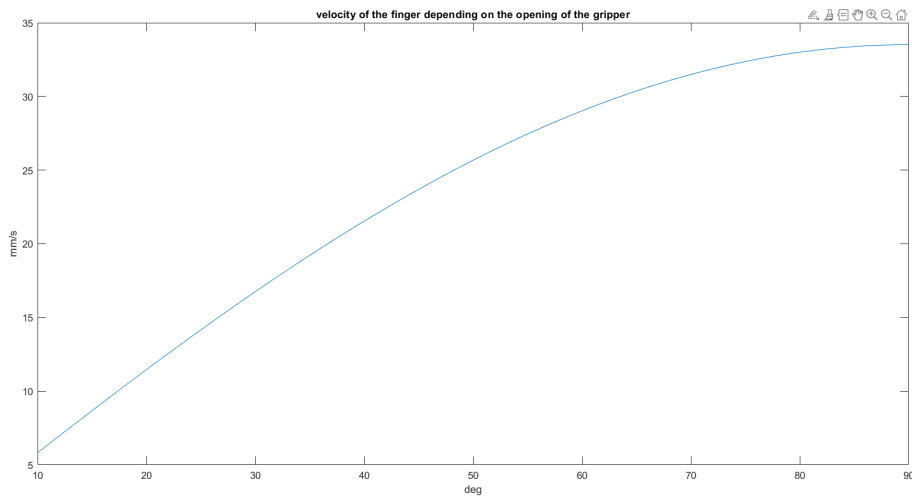


Figure 2.4: Graph displaying the relationship between the gripper opening angle and the horizontal velocity of link 1

The power that the motor should be able to provide is

$$P = 2C_{max}\dot{\theta} = 0.3088 \text{ W} \quad (2.17)$$

We used the motor "Parallax Inc 4 - 6 V Servo Motor, 50 RPM, 27 Ncm Max Output Torque" and selected a gear ratio of 3 because this value allows to satisfy both the requirement on the torque and velocity.

2.1.5 | Robot arm

The motion of the robot arm can be modeled using the forward kinematics, by computing the DH parameters it is possible to calculate the transformation matrices of each link. Combining them it is possible to find the TCP orientation and position in the workspace as a function of the joint angles. The first step is to determine the D-H parameters of the robot arm:

Joint	a_{i-1} (m)	α_{i-1} (rad)	d_i (m)	θ_i (rad)
1	0	0	0.15	θ_1
2	0	$\pi/2$	0	θ_2
3	0.24	π	0	θ_3
4	0.21	0	0.11	θ_4
5	0	$\pi/2$	0.085	θ_5
6	0	$\pi/2$	0	θ_6
7 (TCP)	0.082	0	0	0

Table 2.1: D-H parameters for UR3-e robot arm

The forward pose kinematics is then obtained by deriving the transformation matrices, given by:

$${}^{i-1}_iT = \begin{bmatrix} \cos \theta_i & -\sin \theta_i & 0 & a_{i-1} \\ \sin \theta_i \cos \alpha_{i-1} & \cos \theta_i \cos \alpha_{i-1} & -\sin \alpha_{i-1} & -d_i \sin \alpha_{i-1} \\ \sin \theta_i \sin \alpha_{i-1} & \cos \theta_i \sin \alpha_{i-1} & \cos \alpha_{i-1} & d_i \cos \alpha_{i-1} \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.18)$$

The transformation matrix for the Tool Center Point (TCP) is obtained from the product of the individual transformation matrices:

$${}^0_7T = {}^0_1T {}^1_2T {}^2_3T {}^3_4T {}^4_5T {}^5_6T {}^6_7T \quad (2.19)$$

$${}^0_7T = \begin{bmatrix} r_{11} & r_{12} & r_{13} & {}^0x_6 \\ r_{21} & r_{22} & r_{23} & {}^0y_6 \\ r_{31} & r_{32} & r_{33} & {}^0z_6 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.20)$$

The orthonormal rotation matrix elements for this result are:

$$\begin{aligned} r_{11} &= c_6(s_1s_5 + c_{234}c_1c_5) - s_{234}c_1s_6 \\ r_{12} &= -s_6(s_1s_5 + c_{234}c_1c_5) - s_{234}c_1c_6 \\ r_{13} &= c_5s_1 - c_{234}c_1s_5 \\ r_{21} &= -c_6(c_1s_5 - c_{234}c_5s_1) - s_{234}s_1s_6 \\ r_{22} &= s_6(c_1s_5 - c_{234}c_5s_1) - s_{1234}c_6s_1 \\ r_{23} &= -c_1c_5 - c_{234}s_1s_5 \\ r_{31} &= c_{234}s_6 + s_{234}c_5c_6 \\ r_{32} &= c_{234}c_6 - s_{234}c_5s_6 \\ r_{33} &= -s_{234}s_5 \end{aligned} \quad (2.21)$$

Furthermore, the translational vector components forming the relationship between the position of the origin of TCP with respect to the origin of the base is given by:

$$\begin{aligned} {}^0x_6 &= d_6(c_5s_1 - c_{234}c_1s_5) + d_4s_1 + a_2c_1c_2 + d_5s_{234}c_1 + a_3c_1c_{23} - a_3c_1s_{23} \\ {}^0y_6 &= a_2c_2s_1 - d_4c_1 - d_6(c_1c_5 + c_{234}s_1s_5) + d_5s_{234}s_1 + a_3s_1c_{23} - a_3s_1s_{23} \\ {}^0z_6 &= d_1 + a_3s_{23} + a_2s_2 - d_5c_{234} - d_6s_{234}s_5 \end{aligned} \quad (2.22)$$

These forward pose kinematic equations can be validated by inputting the joint angles of an arbitrary position and verifying the position vector of the TCP from the forward pose kinematics to the cartesian coordinates of the TCP obtained from the robot arm.

2.1.6 | Manipulation and motion planning

Four fixtures are 3D-printed, three for the placement of each part of the trophy before assembly and one for where the trophy is assembled on. The motion is planned to move from fixture one to fixture two and then back again and so on. This is repeated until the whole trophy is assembled. The fixtures are displayed in Appendix A.1.

To carry out the desired motion of the robot arm, the package in Python called *Real Time Data Exchange (RTDE)* is used. Two key features are used for this project and are listed below. These commands are found from the UR3e site [2] and the SDU Robotic's GitLab page [3].

- **getActualQ():** Returns actual joint positions
- **moveJ(q,v,a):** Move to each joint position specified in a path.

To use *moveJ*, the program requires a vector of the desired position. The vector, q , is formed according to equation 2.23. The vector can be defined as a pose, requiring inverse kinematics to determine the corresponding joint positions. Furthermore, the v and a in the *moveJ* function correspond to the velocity and acceleration, respectively.

$$q = [x \quad y \quad z \quad roll(x) \quad pitch(y) \quad yaw(z)] \quad (2.23)$$

2.1.7 | Statics of the robot arm

The static analysis of the robot arm has been developed using the Lagrange approach but considering the kinematic energy equal to zero. This is decided due to the method used in the course literature [1] does not consider gravitational force of the robot arm itself but only external forces applied to the end effector. The following procedure allows modeling of the problem to be formed in a more detailed way, since the external forces are of the same order of magnitude of the gravitational forces of the links of the arm these forces cannot be excluded. From the UR3e site [2] the masses and the position of the center of mass with respect to the reference frame of the same link have been retrieved in Table 2.2.

Mass [kg]	Center of Mass [m]
1.98	[0, -0.02, 0]
3.4445	[0.13, 0, 0.1157]
1.437	[0.05, 0, 0.0238]
0.871	[0, 0, 0.01]
0.805	[0, 0, 0.01]
0.261	[0, 0, -0.02]

Table 2.2: Mass and Center of Mass for each link

The Lagrangian equation for the static analysis is displayed below in equation 2.24, equation 2.25 and equation 2.26.

$$\frac{dU}{d\vec{\theta}} = \vec{\tau} \quad (2.24)$$

$$U = m_1 g^0 h_1 + m_2 g^0 h_2 + m_3 g^0 h_3 + m_4 g^0 h_4 + m_5 g^0 h_5 + m_6 g^0 h_6 \quad (2.25)$$

$$\begin{aligned} {}^0h_1 &= T_1^0 \cdot [0; -0.02; 0; 1]^T \\ {}^0h_2 &= T_2^0 \cdot [0.13; 0; 0.1157; 1]^T \\ {}^0h_3 &= T_3^0 \cdot [0.05; 0; 0.0238; 1]^T \\ {}^0h_4 &= T_4^0 \cdot [0; 0; 0.01; 1]^T \\ {}^0h_5 &= T_5^0 \cdot [0; 0; 0.01; 1]^T \\ {}^0h_6 &= T_6^0 \cdot [0; 0; -0.02; 1]^T \end{aligned} \quad (2.26)$$

Following these steps and differentiating with respect to θ , the τ -values of static torque of the motors are:

$$\tau = \begin{bmatrix} 0 \\ 1.4452 \sin(\theta_2 + \theta_3 + \theta_4) - 7.5189 \cos(\theta_2 + \theta_3) - 13.231 \cos(\theta_2) - 0.65825 \cos(\theta_2 + \theta_3 + \theta_4) \sin(\theta_5) \\ 1.4452 \sin(\theta_2 + \theta_3 + \theta_4) - 7.5189 \cos(\theta_2 + \theta_3) - 0.65825 \cos(\theta_2 + \theta_3 + \theta_4) \sin(\theta_5) \\ 1.4452 \sin(\theta_2 + \theta_3 + \theta_4) - 0.65825 \cos(\theta_2 + \theta_3 + \theta_4) \sin(\theta_5) \\ -0.65825 \sin(\theta_2 + \theta_3 + \theta_4) \cos(\theta_5) \\ 0 \end{bmatrix} \quad (2.27)$$

If we want to add external forces acting on the gripper, that arise from the lifting of the different parts of the trophy, the potential energy should include an additional term

$$U = m_1 g^0 h_1 + m_2 g^0 h_2 + m_3 g^0 h_3 + m_4 g^0 h_4 + m_5 g^0 h_5 + m_6 g^0 h_6 + m_{part} g^0 h_{TCP} \quad (2.28)$$

$${}^0h_{TCP} = T_{TCP}^0 \cdot [0; 0; 0.2; 1]^T \quad (2.29)$$

2.2 | Sensor concept

2.2.1 | Strain gauge

Set-up

The strain gauge was utilized to measure mechanical deformation or strain experienced by the gripper when a component was lifted from the fixture. It operates based on the principle that the electrical resistance of a material changes proportionally to its deformation. The strain gauge was placed in a quarter-bridge configuration to obtain the axial strain of the gripper from which the stress was calculated. The strain gauge had a gauge factor of $2.08 \pm 1\%$ and a gauge resistance of $120 \pm 0.5 \Omega$. The strain gauge was delicately fabricated and the gripper surfaced was prepared before the strain gauge was put on the gripper. This was done to prevent improper handling or placement which can affect the accuracy of the measurements and the reliability of the data. Figure 2.5 shows the strain gauge fabricated on the gripper.

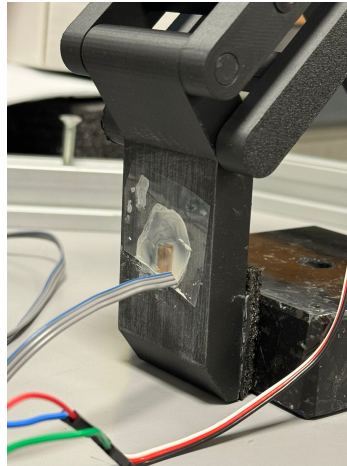


Figure 2.5: Strain Gauge fabricated on the gripper

Data Acquisition System

The data from the strain gauge was acquired using a Data Acquisition System and was visualized using LabVIEW. For our setup the NI 9237 DAQ system was used along with NI9944 for the quarter bridge completion. The NI 9237 is designed to work with strain gauges and can be used to measure the small voltage signals from the quarter-bridge strain gauge with a sampling frequency of 50 kHz. The NI 9237 module features built-in software calibration and can interface directly with the strain gauge without the need for external signal conditioning. Although additional modules can be added for more complex setups. The NI 9944 provides the required excitation voltage to the quarter-bridge circuit, which powers the strain gauge and the Wheatstone bridge. It also ensures that the signal from the strain gauge is properly amplified and filtered, reducing noise, interference and aliasing. The conditioned analog signal is then converted to a digital signal through a 24-bit Analog to Digital converter. Finally, the real time monitoring of the data is done by LabVIEW. Figure 2.6 represents the LabVIEW setup of the strain gauge with the DAQ.

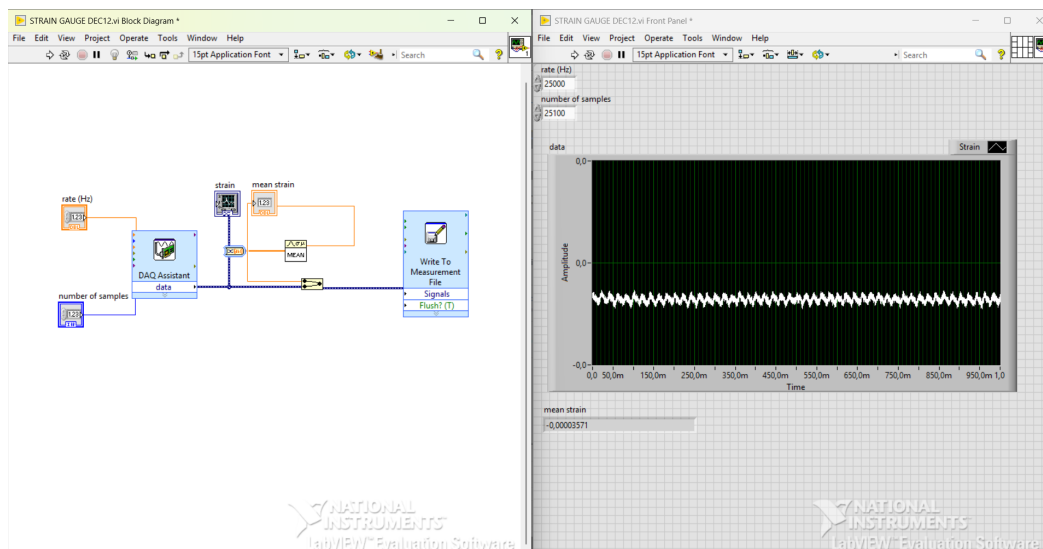


Figure 2.6: LabView model of strain gauge quarter-bridge circuit

Calibration

When the strain gauge is connected to LabVIEW, a window appears where the static strain is measured. To calibrate the strain gauge, this value is re-measured until it measures similar values. These values should be as close to zero as possible. When this is completed, the quarter-bridge is built in the program with the help of the DAQ Assistant module.

Error analysis

The mean strains calculated in LabVIEW are extracted for the scenario where the marble base is gripped by the gripper and lifted in the air. The bending moment, M , can be calculated by using the mean strain, ϵ , moment of inertia of a rectangular cross-section, I , Young's Modulus, E , and half of the thickness of the gripper finger, z . Further, taking the bending moment and dividing by the length between the pin hole joint and where the contact force is approximately applied, L , the contact force, F_K , can be found. The contact force found from the experimental strain value can be compared to the theoretically calculated contact force found and from that any potential error can be analyzed and evaluated.

$$I = \frac{bh^3}{12} \quad (2.30)$$

$$M = \epsilon E \frac{I}{z} \quad (2.31)$$

$$F_K = \frac{M}{L} \quad (2.32)$$

2.2.2 | Torque sensors of the robot arm

The torque sensors in the UR3e robotic arm are located in each joint, enabling precise measurement of forces and torques acting on the robot during operation. These sensors use strain gauge technology to detect small deformations in the joint components caused by applied forces. The sensor data is processed in real time to ensure accurate force and torque estimation. The UR3e sensors are factory-calibrated to ensure high accuracy. Calibration involves applying known forces and torques to the joints and recording the corresponding sensor outputs. This data is used to create calibration matrices stored in the robot's controller. To improve measurement reliability, the UR3e uses digital filtering techniques to process torque sensor data. Low-pass filters reduce noise from vibrations, electrical interference, and other disturbances.

Performance of the torque sensors

Torque Sensor	Torque
Range	± 10.0 Nm
Precision	± 0.1 Nm
Accuracy	± 0.1 Nm

Table 2.3: Performance Characteristics of Force and Torque Sensing

These sensors have been used when the robot arm was in a steady-state condition, measuring the motor torques can be used to verify whether a specific part has been successfully picked up by the gripper. This feature adds an additional layer of reliability to the assembly process, as the robot can detect when a part is missing or improperly placed in the fixture by comparing

the measured motor torques. If the part is not picked up, the assembly process can be halted to avoid further errors.

The torque measurements are performed when the robot stops above the fixtures, immediately after attempting to pick up the parts. The robot data is accessed using a Python program that retrieves the current values of the motor torques, seen Appendix A.2.3. If the part has not been correctly picked up, the gravitational force due to the part's weight will not be applied to the gripper, and the static torques of the robotic arm's motors will reflect this anomaly. An error signal is then triggered, stopping the assembly process. The sensors of the robotic arm have the properties listed in Table 2.3.

2.2.3 | Ultrasonic sensor

An ultrasonic sensor which is of the model HC-SR04 was utilized to detect the open and close position of the gripper. This sensor operates by emitting ultrasonic waves and measuring the time it takes for the waves to reflect back after hitting an obstacle. When attached to the gripper mechanism, the HC-SR04 helps in determining the distance between the sensor and the nearby object, such as the opposing part of the gripper or the object being gripped. By analyzing the distance measurements, the control system can determine whether the gripper is fully open, fully closed, or in an intermediate state. This mechanism is used to stop the motor once it reaches to a particular position so that the motor does not run continuously even after reaching the maximum close and open positions. The ultrasonic sensor is mounted on one side of the gripper as shown in figure 2.7.

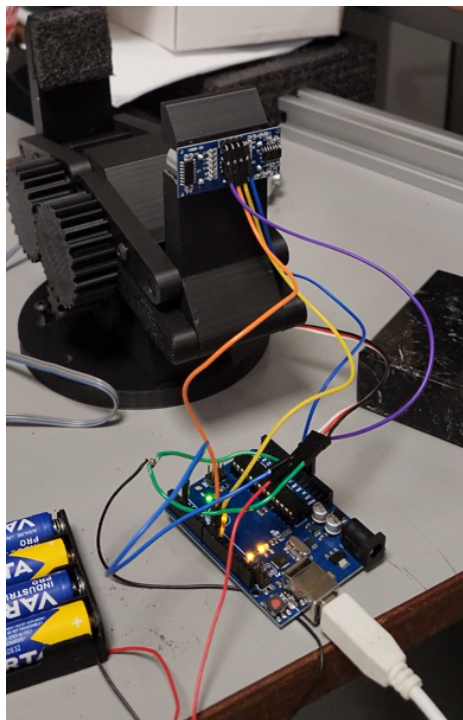


Figure 2.7: Ultrasonic sensor mounted on the gripper

The ultrasonic sensor is connected to the Arduino which is used to control the motor. The code for this can be found in Appendix A.2.1.

3 | Results

3.1 | Robotics concept

3.1.1 | Gripper fabrication

Based on the optimal design parameters for the gripper, a CAD model is created in SOLIDWORKS. The gripper consists of the base which bolts on to the robot arm, a housing which supports the motor and the transmission system, and finally the linkage and fingers of the robot gripper. The fingers are designed as 4-bar linkages on opposite ends of the housing with equal crank lengths to ensure that the fingers always remain straight during the actuation of the gripper. The CAD parameters of the gripper are stated below, along with visuals of the final gripper design (Figure 3.1):

1. Fingers:
 - (a) Length: 70 mm
 - (b) Width: 40 mm
 - (c) Thickness: 13.5 mm
2. Links:
 - (a) Rocker length: 72 mm
 - (b) Crank length: 25 mm
 - (c) Thickness: 10 mm
3. Gear ratio between motor and links: 3

The gripper is fabricated using a 3D-printer with PLA filament. The final assembled gripper with the installed motor and electronics is displayed in Figure 3.2.

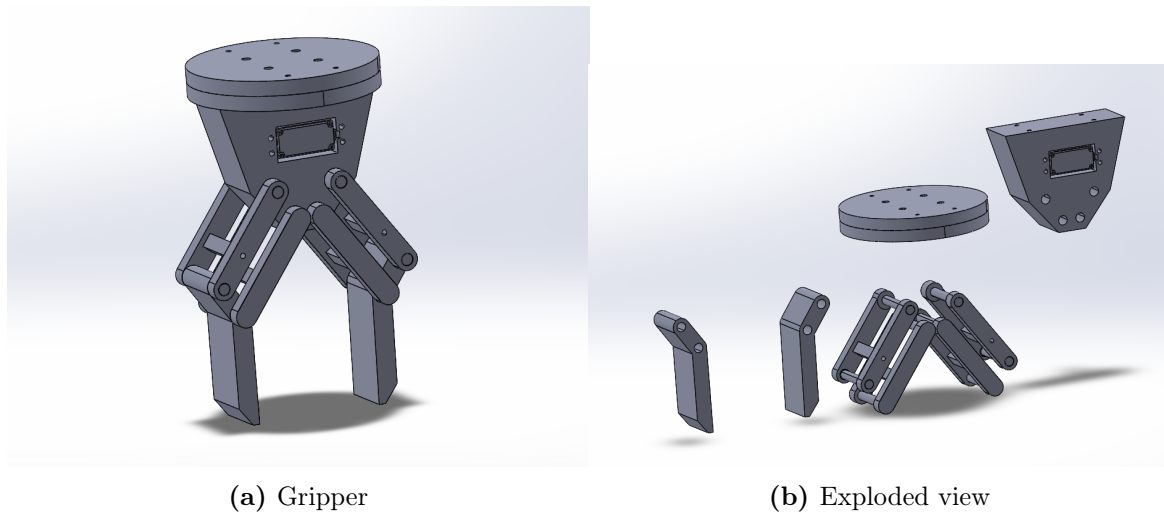


Figure 3.1: CAD model of robot gripper

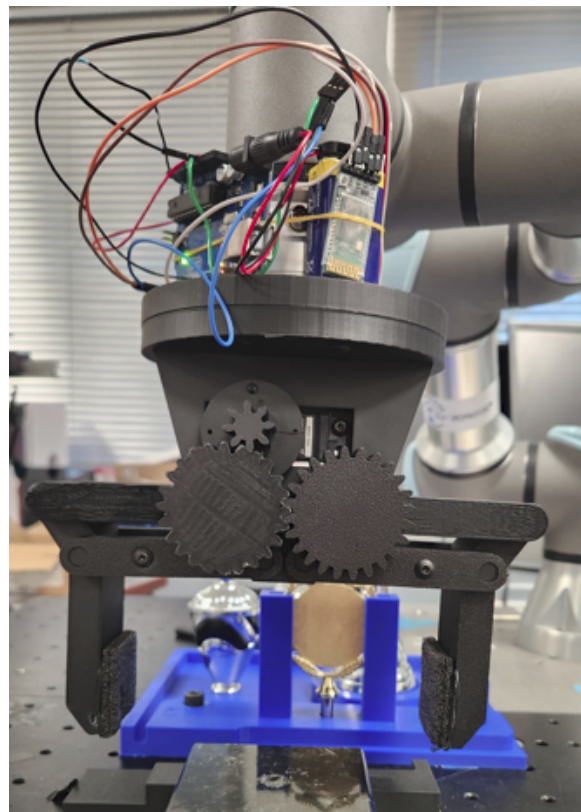


Figure 3.2: Fabricated gripper installed on robot arm

3.1.2 | Gripper manipulation

With the fabricated gripper installed on the base, the next step is to achieve communication with the motor to manipulate the gripper to perform the assembly operation. This is achieved by connecting the motor to an Arduino board, and inputting a program that manipulates the gripper via pulse width modulation. Additionally, the board is connected to an Arduino Bluetooth module HC-05, allowing remote control of the gripper during the assembly operation through an app on the smartphone. The code for manipulating the gripper is available

in Appendix A.2.1.

3.1.3 | Assembly operation

Once remote communication has been achieved with the gripper and the robot arm, the assembly operation is commenced. The pick and drop positions for the robot arm are obtained by moving the robot to the desired positions and extracting the inverse kinematics of the joint positions using the **getActualQ** function in *RTDE* as introduced in section 2.1.6.

The obtained joint angles are then used to move the robot to the desired positions, i.e., from the fixtures holding the trophy components to the assembly fixture, using the **moveJ** function (taking in the desired joint angles, velocity, and acceleration). At the desired positions the gripper is manipulated using a remote control to pick and drop the trophy components. Owing to the geometry of some components, a conscious choice is made to omit assembly of certain components by the robot arm and are instead installed by manual intervention (parts 3,6,8,9,13).

The entire assembly process is recorded and is presented along with this report. The code for the entire assembly of the trophy is available in Appendix A.2.2.

3.2 | Sensor concept

3.2.1 | Strain gauge

The strain values are recorded using the setup and LabVIEW. For this experiment, only part 4 is used to record the strain values. The recorded strain when this part is lifted is $\varepsilon = 1.1516 \cdot 10^{-4}$.

With this strain value obtained, the calculation of the moment is performed using equation 2.31.

The known values are given below and are obtained by measurement and calculations using equation 2.30.

$$\begin{aligned}
 E &= 1.5 \text{ GPa} \\
 z &= 7 \times 10^{-3} \text{ m} \\
 I &= 9.14 \times 10^{-9} \text{ m}^4 \\
 L &= 52 \times 10^{-3} \text{ m}
 \end{aligned}$$

The moment is obtained as: $M = 0.2257 \text{ Nm}$.

Once the bending moment value is obtained, the gripping force can be calculate using equation 2.32.

The experimental gripping force is found as: $F_K = 4.3406 \text{ N}$

The above gripping force is validated against the value obtained using the Gripping force formula. The gripping force is manually calculated using the formula,

$$F_K = \frac{m \cdot (g + a)}{\mu \cdot n} \quad (3.1)$$

where,

Mass of part 4 (the marble base): $m = 558 \times 10^{-3} \text{ kg}$

Gravity: $g = 9.81 \text{ m/s}^2$

Maximum acceleration: $a = 0.1 \text{ m/s}^2$

Coefficient of friction between plastic and marble: $\mu = 0.6$

Number of gripper fingers: $n = 2$

The theoretical gripping force, using these numbers, can be calculated as $F_K = 4.6082 \text{ N}$, which is approximately equal to the gripping force obtained using the strain gauge. Hence, the data is validated through the use of strain gauge. The calculated percentage error between the theoretical and measured gripping forces is approximately 5.81%. This indicates a moderate deviation between the calculated and measured gripping forces. Depending on the application, this error may be acceptable or require further investigation to improve accuracy.

Moreover, a FEM Simulation is done (figure 3.3) on the gripper to study the stress levels on the gripper when the marble base is lifted where it requires a gripping force of 4.6 N .

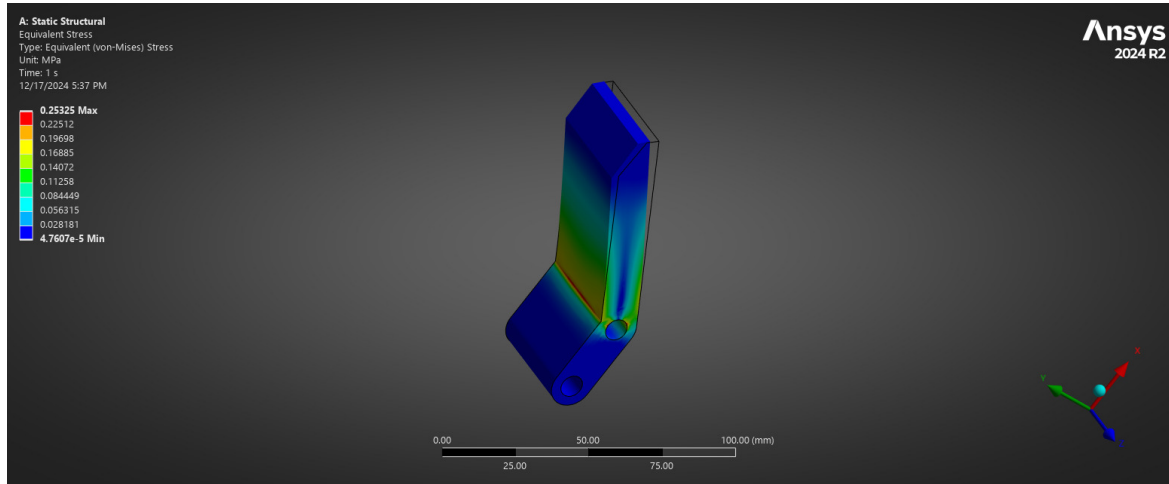


Figure 3.3: FEM Simulation of gripper

It can be inferred from the simulation that the maximum stress experienced by the gripper is around 0.25 MPa near the pin hole joint of the gripper.

3.2.2 | Torque sensors of the robot arm

This section demonstrates an application of the torque sensors to verify if the lift of the marble base is successful or not. Specifically, a comparison between the experimental torque, measured by the robot, when the marble base has been picked up correctly versus when the base is missing in the fixtures and the gripper fails to hold it is analyzed. These torque values have been retrieved in the same configuration of the arm (joint angles = $[-0.8822, -0.2307, -0.5803, -0.7598, -1.5708, 0.6886]$) and are listed in order from the base joint till the last joint.

- When marble is gripped:

$$\vec{\tau} = [0.680961, -0.013554, -0.898821, -0.049999, -0.218371, -0.108412] \quad (3.2)$$

- When the marble is not gripped:

$$\vec{\tau} = [0.138066, 2.833542, 1.18844, 0.490097, -0.171318, 0.135241] \quad (3.3)$$

It is visible that there is a difference between these values due to the missing gravity force on the marble that acts on the end effector when the marble base is not gripped.

Using the torque sensors allows control anomalies of the process and stops the assembly if something goes wrong. It is possible to verify this data through the static computation for the robot arm that is shown in section 2.1.7.

3.2.3 | Ultrasonic sensor

The ultrasonic sensor has been used to increase the level of automatization of the assembly process. A video showing how the motor stops the opening when the maximum distance between the gripper fingers is reached can be found in the folder that contains this paper.

4 | Conclusion

This paper aims to automate the assembly of a trophy, composed of various components with distinct shapes and dimensions, using a UR3e robotic arm and a designed 3D-printed gripper. Throughout the process, several challenges were encountered, including the difficulty in handling specific components due to their geometry, such as the rods and the gold medal. These challenges were addressed by leveraging the flexibility of the robot and gripper design, though certain tasks, such as screwing the rods into place, proved to be less efficient due to the difficulty in applying a torque and due to the last joint angle range limitation.

Despite these challenges, the project demonstrates the feasibility of using the UR3e robot arm in conjunction with a 3D-printed gripper for an easy assembly task.

The integration of sensors in this project significantly enhanced the level of automation. The strain gauge and torque sensors enabled precise assessment of whether a part had been successfully picked up or released by the gripper. Meanwhile, the ultrasonic sensor provided valuable feedback on the gripper's opening position, ensuring accurate control of its motion. Together, these sensors contributed to a more reliable and efficient assembly process, allowing for greater precision and automation throughout the task.

5 | Discussion

A 3D-printed gripper has been developed to collaborate with a robot arm and integrated sensors, facilitating the assembly of a trophy in a semi-automated process. However, certain aspects of the assembly, particularly involving the rods and the gold medal (parts 3, 9, and 13), present challenges. The circular cross-section of the rods and their low coefficient of friction make it difficult for the gripper to apply sufficient torque for screwing operations. To address this, a new design is proposed of a specialized end effector that can be mounted either alongside the gripper or on an additional robot arm, specifically to perform screwing tasks. This enhancement would enable precise and reliable handling of these components, overcoming the limitations of the current gripper. Furthermore, the trophy design itself could be optimized to facilitate robotic assembly by consolidating the double rods and the gold medal into a single unified component. Such a modification would eliminate the need for complex manipulations, significantly accelerating the assembly process and enabling full automation with the gripper alone. By combining hardware enhancements and design optimization, these improvements aim to streamline the assembly workflow, reduce reliance on human intervention, and maximize the efficiency of the robotic system.

Integrating computer vision can be a more sophisticated solution for identifying the position and orientation of parts, eliminating the need for complex fixtures in manufacturing automation systems. By analyzing parts in real-time using cameras and advanced algorithms, computer vision ensures precise alignment and guides the robot. This approach simplifies the design of the robotic assembly system, increases flexibility, and reduces the costs associated with fixture production and maintenance. In addition, it improves efficiency by enabling quick adaptations to varying production needs, ensuring high accuracy and minimizing errors.

Several potential sources of error could impact the performance of the strain gauge and the accuracy of the measurements. Vibrations caused by passing cars, as the setup was located near a road, introduced small disturbances to the system. Additionally, uneven application of adhesive beneath the strain gauge could result in inconsistent stress distribution, affecting the reliability of readings. Calibration was another critical factor, as failing to calibrate the strain gauge after each program run allowed cumulative errors to persist. External electromagnetic interference and fluctuations in the power supply can be a contributing factor to noise in the signal, while misalignment of the strain gauge, such as being off-center, can be a reason for detection of unintended strains. Furthermore, the strain gauge exhibited slight hysteresis and drift over time, which could be a factor of inexact precision of data.

This study can serve as a foundation for further advancements in the automation of heavy industrial equipment, particularly in areas where precision and reduced assembly time are critical. By addressing current challenges and establishing innovative methods, it paves the way for more efficient, reliable, and precise automation solutions, ultimately transforming industrial operations and boosting productivity.

6 | Contributions

Aanand contributed to the project by designing and fabricating the fixtures. This involved developing the CAD model and taking the dimension of the trophy parts. Additionally, he worked on testing the gripping force using strain gauges and validated the results against theoretical calculations.

Alessandro preliminary designed the gripper and computed the static forces on its links, then he optimized some design parameter to meet the constraints imposed by the geometry of the parts and the motor properties. He configured the ultrasonic sensor with the Arduino board and implemented a Bluetooth module to automate the gripper's operation.

Malcolm managed the modeling of the robotic arm and extracting the DH parameters. He was able to develop the forward kinematics and static analysis to help the motion planning and programming of the robot. Additionally he ran an FEM simulation on the gripper to ensure that the stresses during the operation are safe.

Vilma focused on the contact forces and pressures between the gripper and the trophy parts, this helped sizing the motor and gears of the gripper. Moreover she programmed the robotic arm, and developed the motion planning algorithm. She collaborated with Aanand installing the strain gauge and reading the signal.

Bibliography

- [1] J.J. Craig. Introduction to Robotics: Mechanics and Control. Always Learning. Pearson Education, 2014. ISBN: 9781292040042.
- [2] DH Parameters for calculations of kinematics and dynamics.
<https://www.universal-robots.com/articles/ur/application-installation/dh-parameters-for-calculations-of-kinematics-and-dynamics/>. Accessed: 2024-12-23. 2024.
- [3] SDU Robotics. Examples, Universal Robots RTDE C++ Interface.
https://sdurobotics.gitlab.io/ur_rtde/examples/examples.html. Accessed: 2024-12-23.

A | Appendix

A.1 | Appendix 1



Figure A.1: Parts of the trophy to be assembled

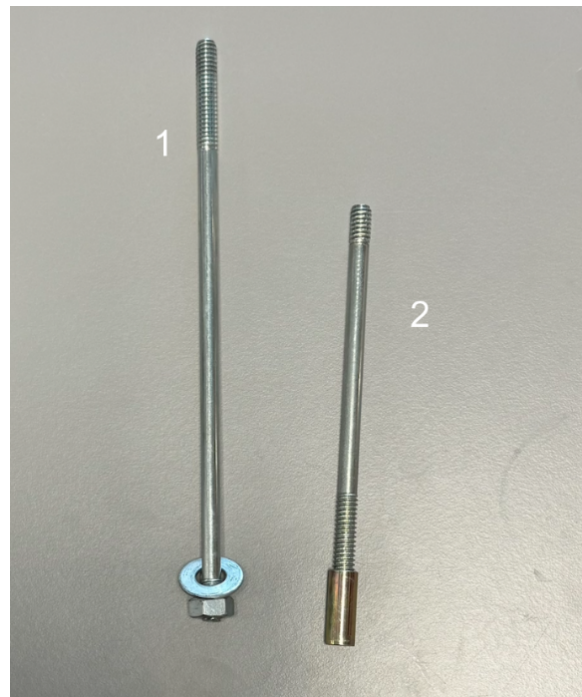


Figure A.2: Pre-assembled parts that are not done by the robot program

A.2 | Appendix 2

A.2.1 | Program to control gripper

```
1 // Define motor control pins
2 const int motor = 3;
3
4 // Variable to store the current motor state
5 char motorState = '1'; // Start with the motor off
6 int trigPin = 9;
7 int echoPin = 10;
8 int ptt;
9 float ptd;
10 float distance;
11
12 void setup() {
13
14     pinMode(trigPin, OUTPUT);
15     pinMode(echoPin, INPUT);
16     Serial.begin(9600);
17
18     // Initialize motor pin as output
19     pinMode(motor, OUTPUT);
20
21     // Initialize Serial communication for Bluetooth
22     Serial.begin(9600); // Ensure this matches the baud rate ...
                           of your Bluetooth module
```

```
23
24 // Turn off the motor initially
25 digitalWrite(motor, LOW);
26 }
27
28 void loop() {
29
30     digitalWrite(trigPin, LOW);
31     delayMicroseconds(10);
32     digitalWrite(trigPin, HIGH);
33     delayMicroseconds(10);
34     digitalWrite(trigPin, LOW);
35
36     ptt = pulseIn(echoPin, HIGH);
37
38
39     ptd = 343.0 * ptt / 1000000; // Corrected assignment and ...
        added 0 to float value
40     distance = ptd * 1000.0 / 2; // Dividing by 2 to get ...
        one-way distance
41
42     Serial.print("Distance=");
43     Serial.print(distance); // Fixed missing closing parenthesis
44     Serial.println(" mm");
45     delay(1);
46     if (Serial.available() > 0) { // Check if data is ...
        available to read
47         char command = Serial.read(); // Read the command sent ...
            from the mobile app
48         if (command == '1' || command == '2' || command == '3') {
49             motorState = command; // Update the motor state based ...
                on the command
50         }
51     }
52
53     // Perform action based on the current motor state
54     switch (motorState) {
55         case '1': // Stop the motor
56             digitalWrite(motor, LOW);
57             break;
58
59         case '2': // Open (1700 microsecond pulse for servo)
60             digitalWrite(motor, HIGH);
61             delayMicroseconds(1700);
62             digitalWrite(motor, LOW);
63             delayMicroseconds(20000); // Rest time for servo pulse
64                 if (distance>120){
65                     motorState='1';
66                 }
67             break;
```

```

68
69     case '3': // Close (1300 microsecond pulse for servo)
70         digitalWrite(motor, HIGH);
71         delayMicroseconds(1300);
72         digitalWrite(motor, LOW);
73         delayMicroseconds(20000); // Rest time for servo pulse
74
75         break;
76
77     default:
78         // Do nothing if an invalid state somehow occurs
79         break;
80 }
81 }

```

A.2.2 | Program for assembly operation

```

1  import math
2  import time
3  import keyboard
4
5  ### Robot IP - look for ours
6  ROBOT_HOST_IP = "192.168.1.254"
7
8  # # move with moveL
9  import rtde_control
10 rtde_c = rtde_control.RTDEControlInterface(ROBOT_HOST_IP)
11
12 #             x,y,z             rpy (roll (x), pitch (y), yaw (z)) -?
13 # The below code controls the movement of the robot with cartesian ...
14 # The following coordinates are a safe starting point for moveL ([0.25, ...
15 # -0.25, 0.35, 3.1416, 0, 0], 0.2, 0.1)
16 # The values does as follows:
17 # X moves planar to the Windows. The positive direction is away from the door
18 # Y moves towards the windows. The positive direction is towards the windows.
19 # Z moves up and down. The positive direction is upwards.
20 # Rotation X
21 # Rotation Y
22 # Rotation Z
23 # Speed
24 # Acceleration
25 #rtde_c.moveJ([0, -math.pi, math.pi/2, -math.pi/2, -math.pi/2, 0], 0.1, 0.02)
26
27 # The below code controls the movement of the robot with joint angles.
28 # The following coordinates are a safe starting point for moveJ ([-0.4, ...
29 # -1.7, -1.2, -2, 1.5, -2], 0.2, 0.1)
30 # Each number is a value for the rotation of each joint (One rotation is ...
31 # 2*Pi)
32 #rtde_c.moveJ([0, -math.pi, math.pi/2, -math.pi/2, -math.pi/2, 0], 0.8, 0.4)
33
34
35 # # Get joint positions
36 import rtde_receive

```

```

37 rtde_r = rtde_receive.RTDEReceiveInterface(ROBOT_HOST_IP)
38
39 # Move to the desired joint positions
40 def move_des(desired_joint_positions):
41     if desired_joint_positions:
42         rtde_c.moveJ(desired_joint_positions, speed=0.7, acceleration=0.25)
43     else:
44         print("Inverse Kinematics solution not found!")
45
46 def wait_for_key(key="space"):
47     print(f"Waiting for '{key}' key...")
48     keyboard.wait(key)
49     print(f"'{key}' key pressed!")
50
51 def djp(tcp_pose, current_joint_positions):
52     desired_joint_positions = rtde_c.getInverseKinematics(tcp_pose, ...
53         current_joint_positions)
54     return desired_joint_positions
55
56 # Calculate IK for the given TCP pose
57 # This function returns joint positions to achieve the desired TCP pose
58 djp_0 = djp([0.2 , 0, 0.4, -2.2552121731514405, 2.1838866669493817, ...
59     -0.06527367547646405], rtde_r.getActualQ())
60 move_desired_0 = move_des(djp_0) # position over marble base
61 djp_1 = djp([0.3993951964494497, -0.2798580232847306, 0.3, ...
62     -2.2552121731514405, 2.1838866669493817, -0.06527367547646405], ...
63     rtde_r.getActualQ())
64 move_desired_1 = move_des(djp_1) # position over marble base
65 time.sleep(2)
66 djp_2 = djp([0.3993951964494497, -0.2791, 0.235, -2.2552121731514405, ...
67     2.1838866669493817, -0.06527367547646405], rtde_r.getActualQ())
68 move_desired_2 = move_des(djp_2) # lower to pick up marble base
69 time.sleep(5)
70 djp_2_1 = djp([0.3993951964494497, -0.2791, 0.27, -2.2552121731514405, ...
71     2.1838866669493817, -0.06527367547646405], rtde_r.getActualQ())
72 move_desired_2_1 = move_des(djp_2_1) # middle lift marble
73 djp_3 = djp([0.3993951964494497, -0.2798580232847306, 0.3, ...
74     -2.2552121731514405, 2.1838866669493817, -0.06527367547646405], ...
75     rtde_r.getActualQ())
76 move_desired_3 = move_des(djp_3) # lift the marble base
77 time.sleep(2)
78 djp_4 = djp([0.2576064929801978, 0.04640318015011849, 0.3, ...
79     2.2191525373307366, -2.218591622854392, 0.025637973761581438], ...
80     rtde_r.getActualQ())
81 move_desired_4 = move_des(djp_4) # move the object to over the assembly ...
82     fixture
83 time.sleep(2)
84 djp_5 = djp([0.2580064929801978, 0.04400318015011849, ...
85     0.23544694205390732, 2.2191525373307366, -2.218591622854392, ...
86     0.025637973761581438], rtde_r.getActualQ())
87 move_desired_5 = move_des(djp_5) # lower to release marble base
88 time.sleep(2)
89 djp_6 = djp([0.2566064929801978, 0.04640318015011849, 0.36, ...
90     2.2191525373307366, -2.218591622854392, 0.025637973761581438], ...
91     rtde_r.getActualQ())
92 move_desired_6 = move_des(djp_6) # position over assembly
93 time.sleep(2)
94 djp_7 = djp([0.20042445314470567, -0.22230969950081841, 0.4, ...
95     2.227291531129319, 2.211939700926393, 0.004941294965979073], ...
96     rtde_r.getActualQ())
97 move_desired_7 = move_des(djp_7) # position over silver cone
98 time.sleep(2)

```

```

82 djp_8 = djp([0.20042445314470567, -0.22230969950081841, ...
    0.3105732586021112, 2.227291531129319, 2.211939700926393, ...
    0.004941294965979073], rtde_r.getActualQ())
83 move_desired_8 = move_des(djp_8) # lower to pick up silver cone
84 time.sleep(5)
85 djp_9 = djp([0.20042445314470567, -0.22230969950081841, 0.45, ...
    2.227291531129319, 2.211939700926393, 0.004941294965979073], ...
    rtde_r.getActualQ())
86 move_desired_9 = move_des(djp_9) # lift the silver cone
87 time.sleep(2)
88 djp_10 = djp([0.25747879790076076, 0.040641858458700525, 0.38, ...
    -2.2242708629727175, 2.19801354240695, -0.003900237233785546], ...
    rtde_r.getActualQ())
89 move_desired_10 = move_des(djp_10) # position over assembly
90 time.sleep(2)
91 djp_11 = djp([0.25747879790076076, 0.040641858458700525, ...
    0.3346326410665209, -2.2242708629727175, 2.19801354240695, ...
    -0.003900237233785546], rtde_r.getActualQ())
92 move_desired_11 = move_des(djp_11) # release the silver cone
93 time.sleep(5)
94 djp_12 = djp([0.25747879790076076, 0.040641858458700525, 0.38, ...
    -2.2242708629727175, 2.19801354240695, -0.003900237233785546], ...
    rtde_r.getActualQ())
95 move_desired_12 = move_des(djp_12) # position over assembly
96 time.sleep(2)
97 djp_13 = djp([0.19702478193746242, -0.32809679131604247, 0.32, ...
    2.1965423453115416, 2.2212823180972756, -0.0025690155116824885], ...
    rtde_r.getActualQ())
98 move_desired_13 = move_des(djp_13) # over 7
99 djp_14 = djp([0.19702478193746242, -0.32809679131604247, ...
    0.28007255634863637, 2.1965423453115416, 2.2212823180972756, ...
    -0.0025690155116824885], rtde_r.getActualQ())
100 move_desired_14 = move_des(djp_14) # pick up 7
101 time.sleep(5)
102 wait_for_key()
103 djp_14_extra = djp([0.19702478193746242, -0.32809679131604247, 0.33, ...
    2.1965423453115416, 2.2212823180972756, -0.0025690155116824885], ...
    rtde_r.getActualQ())
104 move_desired_14_extra = move_des(djp_14_extra) # pick up 7 - midpoint
105 time.sleep(1)
106 djp_15 = djp([0.19702478193746242, -0.32809679131604247, 0.40, ...
    2.1965423453115416, 2.2212823180972756, -0.0025690155116824885], ...
    rtde_r.getActualQ())
107 move_desired_15 = move_des(djp_15) # over 7
108 time.sleep(2)
109 djp_16 = djp([0.15729708565994466, 0.07529596822279454, ...
    0.5397906324507137, 0.9836905137923578, 2.5516037867521386, ...
    0.033546989516269714], rtde_r.getActualQ())
110 move_desired_16 = move_des(djp_16) # over the assembly 1
111 time.sleep(2)
112 djp_17 = djp([0.17505804656231835, 0.06696153048030926, ...
    0.5291249914694922, 1.0140421723926838, 2.6234208935073546, ...
    0.04973405063045551], rtde_r.getActualQ())
113 move_desired_17 = move_des(djp_17) # over the assembly 2
114 time.sleep(1)
115 djp_18 = djp([0.22539528476526643, 0.04269787460505869, ...
    0.48762654806036637, 1.0968896745073649, 2.81089995933692, ...
    0.08695115009492947], rtde_r.getActualQ())
116 move_desired_18 = move_des(djp_18) # over the assembly 3
117 time.sleep(1)
118 djp_19 = djp([0.2434960615790283, 0.040848531102251735, ...
    0.46984572941837527, 1.0735570524841662, 2.894058285295974, ...

```

```

    0.08193520834912037], rtde_r.getActualQ())
119 move_desired_19 = move_des(djp_19) # over the assembly 4
120 time.sleep(1)
121 djp_20 = djp([0.25409476151004556, 0.04581500146278324, ...
    0.402155168138056, 0.9950274948254809, 2.9704537619835407, ...
    -0.000403012645250943347], rtde_r.getActualQ())
122 move_desired_20 = move_des(djp_20) # over the assembly 5
123 time.sleep(5)
124 djp_21 = djp([0.18585601547611658, 0.07095327898161854, ...
    0.5195823794255111, 0.9150524581737594, 2.7172290619047765, ...
    -0.009830444741404886], rtde_r.getActualQ())
125 move_desired_21 = move_des(djp_21) # over the assembly 6
126 time.sleep(2)
127 djp_22 = djp([0.48488530990958645, -0.018134769604416075, 0.3, ...
    -2.222768539769225, -2.17812360180389, 0.003162475426177084], ...
    rtde_r.getActualQ())
128 move_desired_22 = move_des(djp_22) # over the bottom cone
129 time.sleep(2)
130 djp_23 = djp([0.48488530990958645, -0.018134769604416075, ...
    0.24858366515404673, -2.222768539769225, -2.17812360180389, ...
    0.003162475426177084], rtde_r.getActualQ())
131 move_desired_23 = move_des(djp_23) # pick up the bottom cone
132 wait_for_key()
133 djp_24 = djp([0.48488530990958645, -0.018134769604416075, 0.3, ...
    -2.222768539769225, -2.17812360180389, 0.003162475426177084], ...
    rtde_r.getActualQ())
134 move_desired_24 = move_des(djp_24) # over the bottom cone
135 time.sleep(2)
136 djp_25 = djp([0.1972687614404333, -0.023440688964192956, ...
    0.5586570412427576, 2.154632732654807, 1.2928285160644182, ...
    -0.031734001770562806], rtde_r.getActualQ())
137 move_desired_25 = move_des(djp_25) # midpoint path to assembly
138 time.sleep(2)
139 djp_26 = djp([0.18033987898246495, 0.044488578873292536, ...
    0.524042139397563, 2.4370103336078404, 1.618674304477491, ...
    0.4338039661314916], rtde_r.getActualQ())
140 move_desired_26 = move_des(djp_26) # over assembly 1
141 time.sleep(2)
142 djp_27 = djp([0.21203757843278956, 0.018991819640883666, ...
    0.4974335540474018, 2.378326924692783, 1.9565972958639002, ...
    0.3541565962231168], rtde_r.getActualQ())
143 move_desired_27 = move_des(djp_27) # over assembly 2
144 time.sleep(2)
145 djp_28 = djp([0.2456498306918776, 0.04847669456620357, ...
    0.45065654329564436, 2.104896844802446, 2.2393604085317325, ...
    0.07741838747406024], rtde_r.getActualQ())
146 move_desired_28 = move_des(djp_28) # over assembly 3
147 time.sleep(5)
148 djp_29 = djp([0.18033987898246495, 0.044488578873292536, ...
    0.524042139397563, 2.4370103336078404, 1.618674304477491, ...
    0.4338039661314916], rtde_r.getActualQ())
149 move_desired_29 = move_des(djp_29) # over assembly 4
150 time.sleep(2)

```

A.2.3 | Program for retrieving the torques from the robot

```

1 import socket
2 import rtde.rtde_config as rtde_config
3 # Robot configuration
4 robot_ip = "192.168.1.254" # Replace with your robot's IP

```

```

5 robot_port = 30002          # Port for URScript commands
6
7 # PC configuration
8 pc_ip = "192.168.1.224"     # Replace with your PC's IP
9 pc_port = 50000             # Port for receiving torque data
10 timeout_seconds = 10       # Automatic stop timeout (in seconds)
11
12 try:
13     # Start a listening server on the PC to receive torque data
14     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
15         server.bind((pc_ip, pc_port)) # Bind to your PC's IP and port
16         server.listen(1)
17         server.settimeout(timeout_seconds) # Stop automatically after ...
18         print(f"Listening for torque data on {pc_ip}:{pc_port}...")
19
20     # Send the URScript command to the robot
21     with socket.create_connection((robot_ip, robot_port), timeout=5) ...
22         as client:
23         # URScript command to send joint torques back to the PC
24         command = (
25             f"def send_joint_torques():\n"
26             f"    socket_open(\"{pc_ip}\", {pc_port})\n"
27             f"    joint_torques = get_joint_torques()\n"
28             f"    socket_send_string(to_str(joint_torques))\n"
29             f"    socket_close()\n"
30             f"end\n"
31         )
32         print("Sending URScript command to robot...")
33         client.sendall(command.encode('utf-8'))
34
35     # Accept a connection from the robot
36     try:
37         conn, addr = server.accept()
38         with conn:
39             print(f"Connected by robot at {addr}")
40             # Receive and display torque data
41             data = conn.recv(1024)
42             print("Torque Data:", data.decode('utf-8'))
43     except socket.timeout:
44         print(f"No data received within {timeout_seconds} seconds. ...
45             Stopping the server.")
46
47 except Exception as e:
48     print(f"An error occurred: {e}")
49
50 # Load configuration
51 config_file = "control_configuration.xml"
52 conf = rtde_config.ConfigFile(config_file)
53 output_names, output_types = conf.get_recipe("output")
54
55 # Connect to the robot
56 con = rtde.RTDE(ROBOT_HOST, ROBOT_PORT)
57 con.connect()
58
59 # Setup output recipe
60 con.send_output_setup(output_names, output_types)
61
62 # Start data synchronization
63 if not con.send_start():
64     print("Unable to start synchronization")

```

```
64     exit()
65
66 try:
67     while True:
68         # Receive the data package
69         state = con.receive()
70         if state is None:
71             break
72
73         # Extract joint currents
74         joint_currents = state.actual_current
75         print("Joint Currents:", joint_currents)
76
77 finally:
78     # Stop synchronization and disconnect
79     con.send_pause()
80     con.disconnect()
```